

FORMATION EMBARQUÉE

TD 08 Schneider

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TD 08 — Schneider : énoncés et corrigés détaillés

Cible : Machine Expert Basic (M221 simulé) pour les exercices 1 et 3, Control Expert (ou CODESYS) pour le ST et le SFC. Les concepts du TD 07 sont supposés acquis — on ne re-explique pas l'auto-maintien.

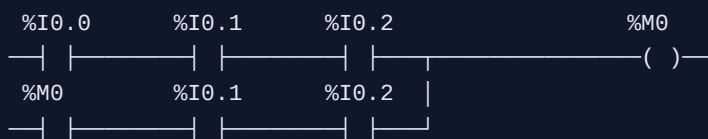
Exercice 1 — Démarrage étoile/triangle

Énoncé. Moteur en étoile/triangle : au départ, contacteur ligne **KM1** + contacteur étoile **KM2** pendant 5 s, puis passage en triangle **KM3**. Verrouillage KM2/KM3 impératif. Boutons **bp_marche** (%I0.0), **bp_arret** (%I0.1, NF câblé), défaut thermique **f_therm** (%I0.2, NF câblé).

Corrigé (LADDER M221)

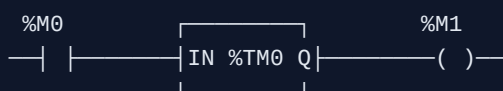
Objets : **%M0** = marche demandée ; **%TM0** : type TON, pré-réglage 5 s (**%TM0.P := 50** avec base 100 ms) ; **%M1** = tempo écoulée.

Réseau 1 — Auto-maintien de la demande :



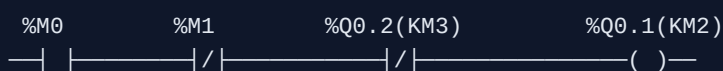
(arrêt et thermique en NF câblé : contact fermé = tout va bien.)

Réseau 2 — Temporisation : **%M0** lance **%TM0** (TON, 5 s) ; sortie du bloc → **%M1**.

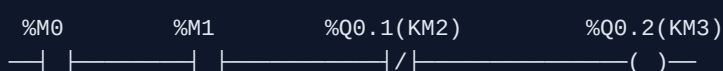


Réseau 3 — Contacteur ligne : **%Q0.0 (KM1) := %M0**.

Réseau 4 — Étoile (KM2) : marche ET tempo non écoulée ET triangle retombé :



Réseau 5 — Triangle (KM3) : marche ET tempo écoulée ET étoile retombée :



Points de correction : 1. Verrouillage croisé KM2/KM3 dans le programme ET, en vrai, aussi par contacts auxiliaires câblés : étoile + triangle fermés ensemble = court-circuit franc. Le logiciel ne suffit jamais seul pour ce risque. 2. Le passage étoile → triangle transite par un état où les deux sont ouverts

(chaque bobine attend la retombée de l'autre) — c'est voulu. 3. Arrêt et thermique coupent tout **via la demande** `%M0` : une seule chaîne d'arrêt, pas trois copies.

Exercice 2 — Bloc `Alternance_Pompes` en ST

Énoncé. Deux pompes ; à chaque demande, démarrer la moins utilisée (compteurs d'heures) ; si la pompe choisie est en défaut, basculer sur l'autre.

Corrigé (DFB Control Expert / FB CODESYS)

Interface : Inputs `demande` , `default_p1` , `default_p2` (Bool) ; Outputs `cmd_p1` , `cmd_p2` , `alarme_aucune_pompe` (Bool) ; Statics `heures_p1` , `heures_p2` (DINT, en secondes), `demande_prec` (Bool), `choix_p1` (Bool), `tick` (instance de TON).

```
(* 1. Choix au FRONT MONTANT de la demande : on ne rebascule pas
   en cours de fonctionnement au fil des heures qui s'accumulent *)
IF demande AND NOT demande_prec THEN
    choix_p1 := (heures_p1 <= heures_p2);
END_IF;
demande_prec := demande;

(* 2. Report automatique sur défaut de la pompe choisie *)
IF demande THEN
    IF choix_p1 AND default_p1 AND NOT default_p2 THEN
        choix_p1 := FALSE;
    ELSIF NOT choix_p1 AND default_p2 AND NOT default_p1 THEN
        choix_p1 := TRUE;
    END_IF;
END_IF;

(* 3. Commandes — une pompe en défaut n'est JAMAIS commandée *)
cmd_p1 := demande AND choix_p1 AND NOT default_p1;
cmd_p2 := demande AND NOT choix_p1 AND NOT default_p2;
alarme_aucune_pompe := demande AND default_p1 AND default_p2;

(* 4. Comptage d'usure : +1 s par seconde de marche, via un TON auto-relancé *)
tick(IN := NOT tick.Q, PT := t#1s);
IF tick.Q THEN
    IF cmd_p1 THEN heures_p1 := heures_p1 + 1; END_IF;
    IF cmd_p2 THEN heures_p2 := heures_p2 + 1; END_IF;
END_IF;
```

Points de correction : le choix figé au front (sinon les pompes « papillonnent » quand les compteurs se croisent) ; le défaut qui inhibe la commande même si la logique de choix se trompait ; l'alarme « aucune pompe disponible » ; les compteurs en Static (rémanents à déclarer si l'usure doit survivre à une coupure — le préciser vaut des points).

Exercice 3 — Table d'échange Modbus + lecture Python

Énoncé. Documenter une table de 10 mots (états, mesures, commandes, mot de vie) et la lire/écrire depuis un PC.

Corrigé

La table (le livrable principal — c'est un document contractuel) :

Mot	Adresse Modbus	Sens	Contenu	Codage
%MW100	100	API → PC	Mot d'état : b0=auto, b1=marche, b2=défaut, b3=niveau haut	bits
%MW101	101	API → PC	Niveau cuve	0..1000 = 0..100,0 % (×10)
%MW102	102	API → PC	Débit pompe	L/min ×10
%MW103	103	API → PC	Code défaut	0=aucun, 1=thermique, 2=niveau...
%MW104	104	API → PC	Mot de vie : compteur +1/s	0..65535, boucle
%MW105	105	PC → API	Mot de commande : b0=marche, b1=acquit défauts	bits
%MW106	106	PC → API	Consigne niveau	% ×10, borné 0..1000
%MW107-109	107-109	—	Réserve (prévoir l'extension)	0

Conventions à énoncer : valeurs analogiques en **entiers ×10** (pas de flottants — un Real occuperait 2 mots avec un ordre dépendant de l'équipement) ; zones API → PC et PC → API **disjointes** (jamais les deux côtés écrivains du même mot) ; le **mot de vie** permet au PC de détecter un automate figé même quand la liaison TCP reste « ouverte ».

Côté PC (pymodbus ≥ 3.x) :

```

from pymodbus.client import ModbusTcpClient
import time

client = ModbusTcpClient("192.168.1.50", port=502)
client.connect()

vie_prec = None
for _ in range(5):
    rr = client.read_holding_registers(address=100, count=5)
    if rr.isError():
        print("erreur Modbus :", rr)
        break
    etat, niveau, debit, default, vie = rr.registers
    print(f"marche={bool(etat & 0x02)} default={bool(etat & 0x04)} "
          f"niveau={niveau/10:.1f}% vie={vie}")
    if vie_prec is not None and vie == vie_prec:
        print("ALERTE : mot de vie figé, automate en panne ?")
    vie_prec = vie
    time.sleep(1)

client.write_register(address=106, value=750) # consigne 75,0 %
client.write_register(address=105, value=0x0001) # bit marche
client.close()

```

Points de correction : la table documentée avec codages et bornes (sans elle, l'exercice vaut zéro : Modbus ne transporte que des mots — le sens est dans le document) ; la surveillance du mot de vie ; le test `isError()` .

Exercice 4 — Cycle de lavage en SFC (Control Expert)

Énoncé. Prélavage 2 min → lavage 5 min → rinçage ×2 → essorage 3 min ; pause/reprise ; abandon avec vidange sécurisée.

Corrigé (structure du SFC + mécanismes clés)



Mécanismes demandés :

- **Compteur de rinçages** : incrémenté à l'entrée de l'étape 4 (action au franchissement, qualificatif P1 chez Schneider), la divergence en OU après vidange reboucle si `nb_rincages < 2`.
- **Pause/reprise** : ne PAS ajouter des transitions « pause » partout — on fige les actions : chaque sortie est conditionnée par `NOT pause` (les actions de niveau `N` passent par une logique commune), et les temporisations utilisent des TON dont l'entrée `IN` est `etape_active AND NOT pause` : la tempo se **suspend** avec la pause.
- **Abandon** : une **séquence dédiée** (étapes 90-91 : arrêt chauffage et moteur → vidange jusqu'à `niveau_bas` → retour à 0), atteinte depuis n'importe où. En SFC Control Expert, l'abandon se traite proprement par un forçage de chaîne ou une transition source depuis une étape englobante ; l'implémenter par « IF abandon THEN etape := 90 » hors du CASE est aussi accepté en version ST.
- **Sécurité transversale hors SFC** : porte ouverte ⇒ moteur et chauffage coupés **immédiatement**, quelle que soit l'étape — cette ligne vit en logique câblée/LADDER en aval des sorties, pas dans la séquence. On ne confie jamais une coupure de sécurité à une séquence qui peut être bloquée dans une étape.

Points de correction : abandon = séquence (on ne « téléporte » pas à Init avec la cuve pleine d'eau chaude) ; tempos suspendues en pause ; sécurité porte hors séquence ; compteur remis à zéro à l'étape 0.

Auto-évaluation avant le TP 4

Sans notes : pourquoi le verrouillage étoile/triangle doit exister aussi en câblé ; les 4 zones de données Modbus et leurs codes fonction ; à quoi sert un mot de vie ; pourquoi une pause ne s'implémente pas en dupliquant des transitions ; où vit une coupure de sécurité (et où elle ne vit pas).

 Passe au [TP 4 — Gestion de cuve](#).